

7장_트랜스포머

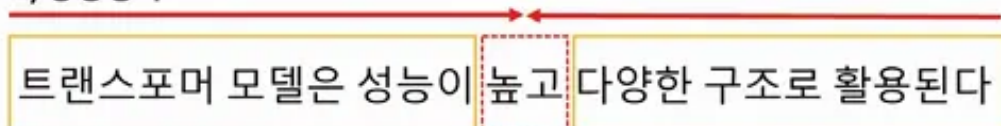
트랜스포머 개요

- 'Attention is All You Need' 논문
- 병렬로 입력 시퀀스 처리 → 긴 시퀀스 처리 효율성: 트랜스포머 모델 >> 순환 신경망
- **셀프 어텐션(Self-Attention)** 기반 → 재귀, 합성곱 연산 없이 입력 토큰 간 관계 모델링 가능
- 대용량 데이터셋을 사용할 때 효율적
 - 광범위 자연어 처리 작업
 - 장기적인 종속성을 포함하는 작업 ex) 기계 번역, 언어 모델링, 텍스트 요약

트랜스포머 기반 모델 구조 - 인코더 & 디코더

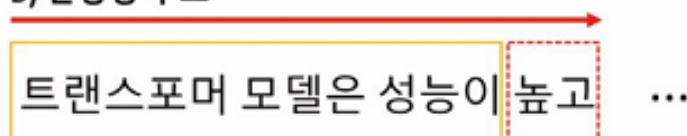
- **오토 인코딩(Auto-Encoding)**: 랜덤하게 문장 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업(task) 수행
 - **인코더(encoder)** - 양방향 구조: 빈칸 토큰의 양옆에 있는 토큰을 참조하는 구조

A) 양방향 구조



- A 입력: 트랜스포머 모델은 성능이 , 다양한 구조로 활용된다
A 출력: 높고
- **자기 회귀(Auto Regressive)**: 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업
 - **디코더(decoder)** - 단방향 구조: 빈칸 토큰의 왼쪽에 있는 토큰들만 참조하는 구조

B) 단방향 구조



- B 입력: **트랜스포머 모델은 성능이**
- B 출력: **높고**

표 7.1 트랜스포머 모델 구조

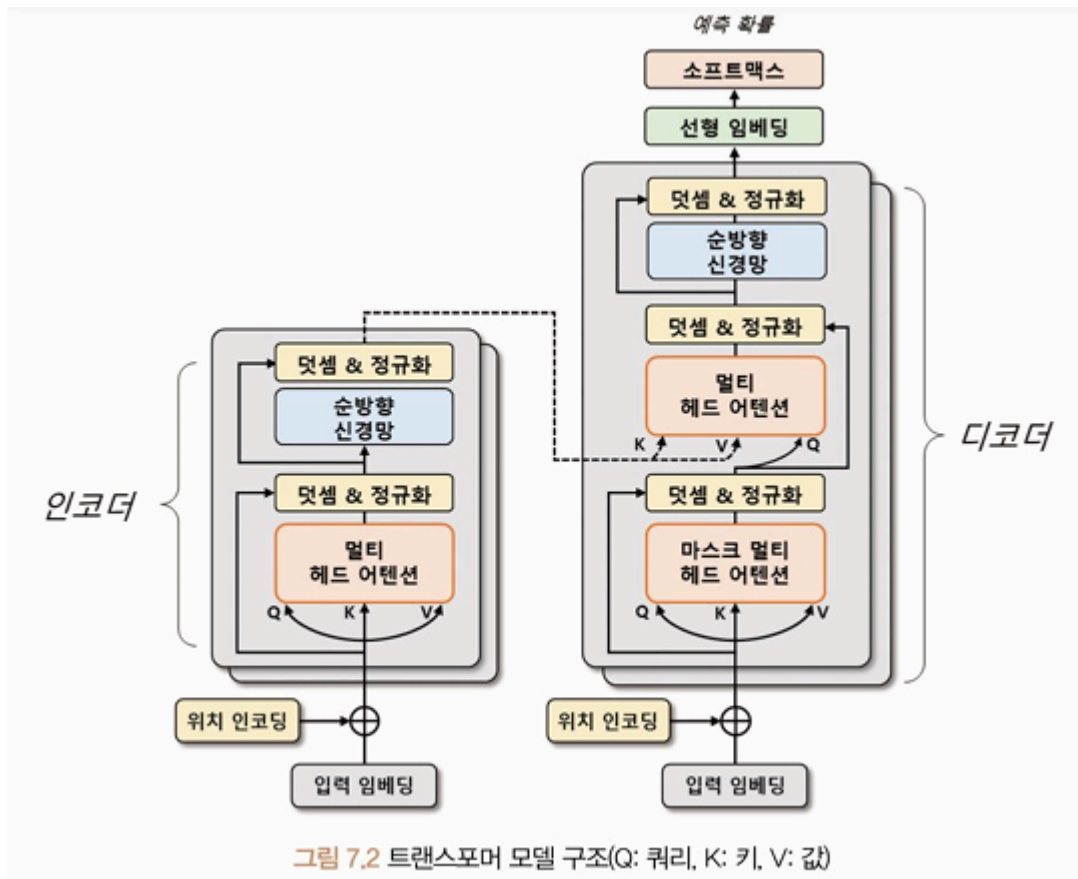
모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

- ELECTRA: 인코더 + 생성적 적대 신경망 판별기 활용한 모델
- T5, BART: 다양한 자연어 처리 작업

Transformer

- 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 활용되는 모델
- **어텐션 메커니즘(Attention Mechanism)**만으로 시퀀스 임베딩 표현
 - 인코더 & 디코더 상호작용
 - 인코더: 입력 시퀀스 임베딩 → 고차원 벡터로 변환
 - 디코더: 인코더 출력 → 출력 시퀀스 생성
 - 어텐션 메커니즘은 입력 시퀀스(인코더 부분)의 각 단어가 출력 시퀀스(디코더 부분)의 어떤 단어와 관련이 있는지 상관관계 계산
 - 번역, 요약문 생성 등 작업 수행
- 학습 속도가 빠르고 병렬 처리 가능 → 대규모 데이터셋에서 높은 성능
- 문장의 길이가 길어져도 성능 유지 - 임베딩 과정에서 문장 전체 정보 고려하기 때문

트랜스포머 구조



- 인코더, 디코더: 각각 N개의 **트랜스포머 블록**으로 구성됨
- 트랜스포머 블록: multi-head attention + 순방향 신경망
 - **Multi-Head Attention**
 - 입력 시퀀스에서 **쿼리, 키, 값 벡터**를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
 - 쿼리와 각 키의 유사도 계산 → 유사도를 가중치로 사용해 값 벡터 합산 → 어텐션 행렬
 - 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터 대체
⇒ 입력 시퀀스 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신하는 것
 - **순방향 신경망**
 - 임베딩 벡터 고도화
 - 입력 벡터에 가중치를 곱하고 편향을 더하며 활성화 함수를 적용하는 여러 선형 계층으로 구성
→ 학습된 가중치들은 입력 시퀀스 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신

- 입력 시퀀스 데이터 → Source & Target 데이터로 나눠 처리
 - ex) 영어 → 한글 번역: 영어-source, 한글-target
- 인코더: 소스 시퀀스 데이터 → 위치 인코딩(Positional Encoding)된 입력 임베딩으로 표현 ⇒ 출력 벡터 생성
- 디코더: **Masked Multi-Head Attention**을 사용해 타겟 시퀀스 데이터를 순차적 생성
 - 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보 참조
 - 최종 디코더 출력 벡터 → 선형 임베딩으로 재표현 ⇒ 이미지, 자연어 모델에 활용

입력 임베딩과 위치 인코딩

입력 시퀀스 단어 - 위치 인코딩 → 벡터 형태로 변환됨

위치 인코딩

입력 시퀀스의 순서 정보를 모델에 전달하는 방법

- 각 단어의 위치 정보를 나타내는 벡터를 더해서 임베딩 벡터에 위치 정보 반영
 - 인코딩할 때 위치 정보를 이용하는 이유?

: 트랜스포머 모델은 입력 시퀀스를 병렬 구조로 처리하기 때문에 순서 정보 제공 X
→ 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영하고자 하는 것
- 각 토큰 위치를 각도로 표현 → sin, cos 함수로 위치 인코딩 벡터 계산
⇒ 입력 시퀀스의 순서 정보 보존 가능

위치 인코딩 계산 과정

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

```
# 위치 인코딩
import math
import torch
```

```

from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        # d_model: 임베딩 차원
        # max_len: 최대 시퀀스
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1) # 2차원 벡터 생성
        div_term = torch.exp( # 스케일링 인자 - 각 위치에 대한 주기적인 신호 생성
            torch.arange(0, d_model, 2) * (-math.log(10000.0)/d_model)
        )

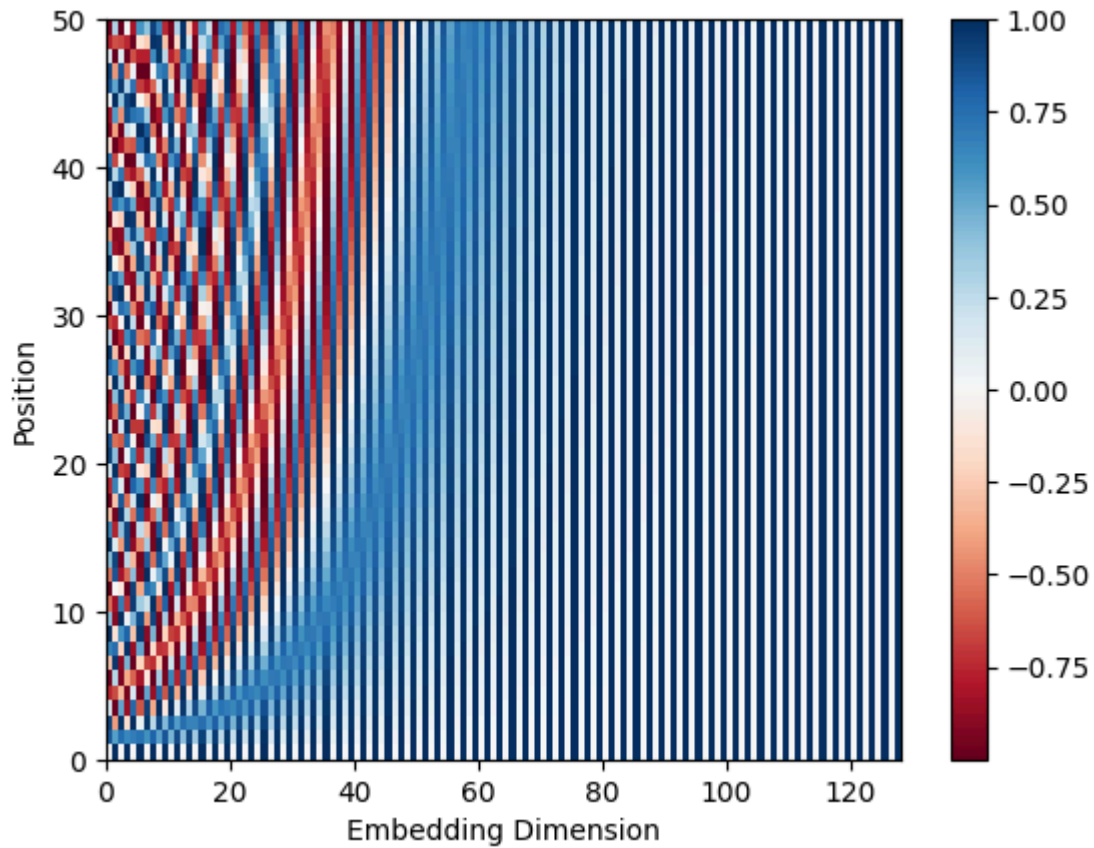
        # 입력 차원과 같은 차원의 positional embedding 생성
        # pe = 위치 정보 -> 나중에 입력 시퀀스에 합산해야 함
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position*div_term) # 홀수 - sin
        pe[:, 0, 1::2] = torch.cos(position*div_term) # 짝수 - cos
        ## pe 차원 - [50, 1, 128] ([최대 시퀀스, 1, 입력 임베딩 차원])
        self.register_buffer("pe", pe) # 모델이 매개변수를 갱신하지 않게 함

    def forward(self, x):
        x = x + self.pe[:x.size(0)] # 위치 정보 합산
        return self.dropout(x)

encoding = PositionalEncoding(d_model=128, max_len=50)

plt.pcolormesh(encoding.pe.numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimension")
plt.xlim((0, 128))
plt.ylabel("Position")
plt.colorbar()
plt.show()

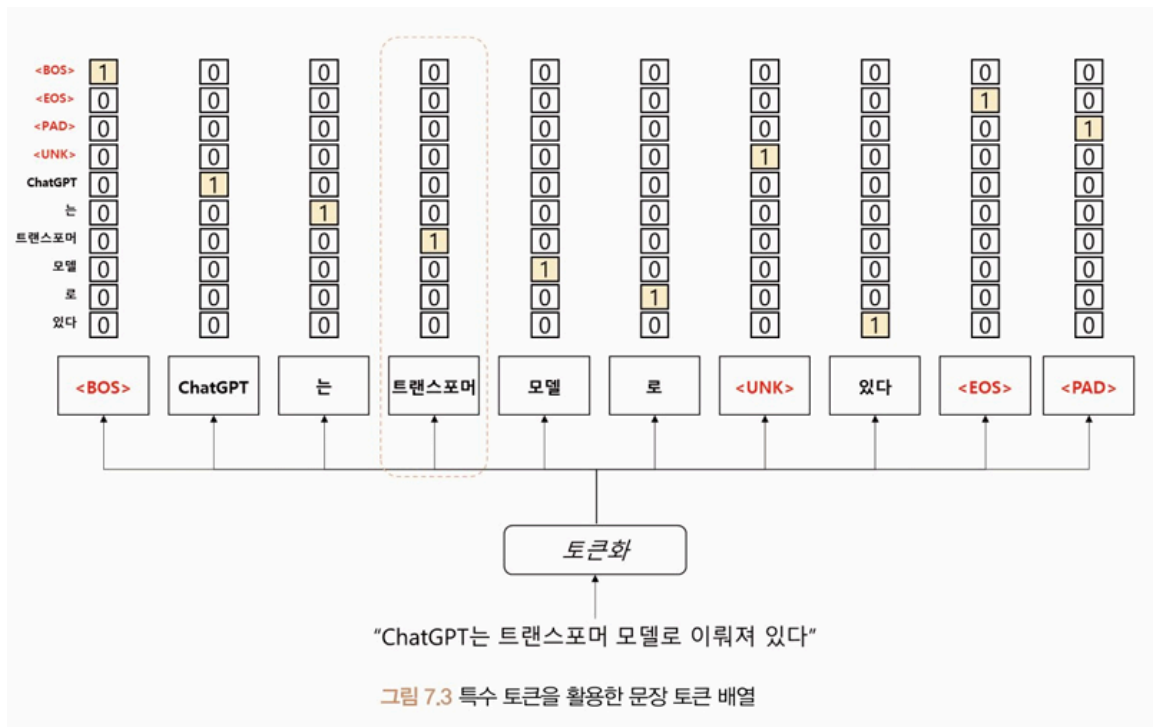
```



→ 어떻게 읽어야 하는지 잘 모르겠다

특수 토큰

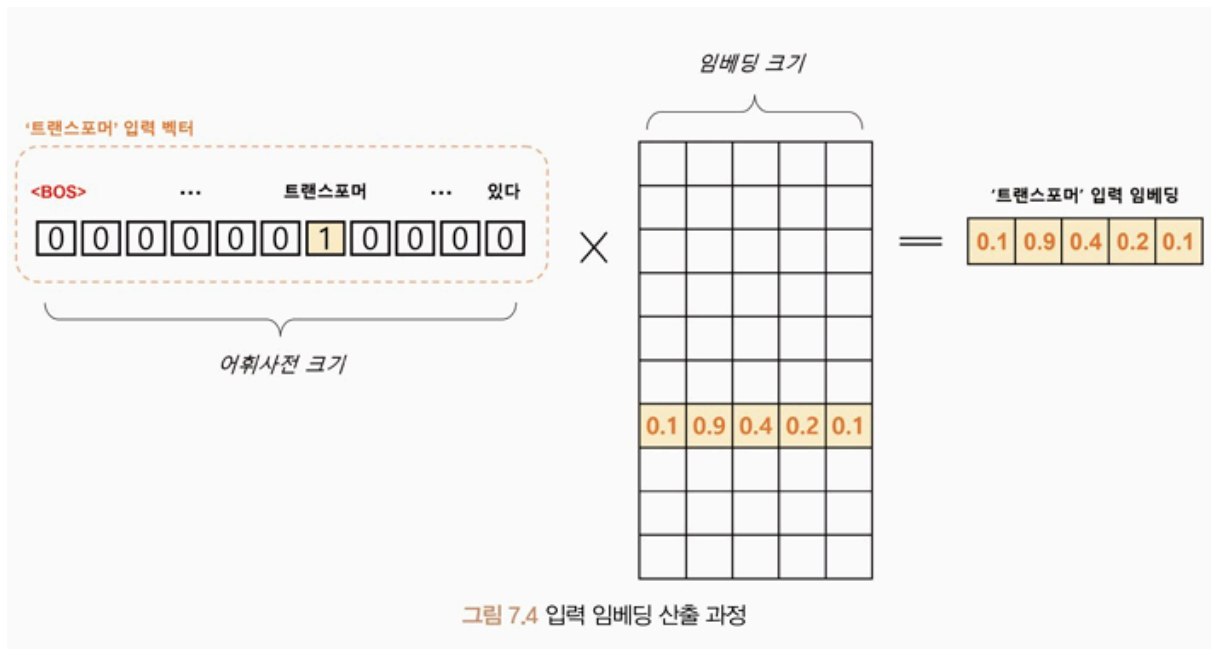
- 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 함
- 마스킹(Masking) 영역을 설정해 일부 입력 무시
 - ex) 번역 모델: 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만 참조
- ex)



- BOS(Beginning of Sentence): 문장의 시작
EOS(End of Sentence): 문장의 끝
- UNK(Unknown): 어휘 사전에 없는 단어 - 모델이 이전에 본 적 없는 단어를 처리할 때 사용
- PAD(Padding): 모든 문장을 일정한 길이로 맞추기 위해 사용

입력 임베딩 변환

: 생성한 문장 토큰 배열 → 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현



- 어휘 사전 크기 V , 입력 임베딩 차원 $d \rightarrow$ '트랜스포머'의 원-핫 벡터: $[1, V]$ 크기 $\rightarrow$ 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환 (lookup)
 - 원-핫 벡터: 어휘 사전에서 해당 단어가 위치한 곳만 1, 나머지 0
- N 개 문장이 최대 S 개의 토큰 길이를 가질 때:
 - $[N, S, V]$ 크기의 원-핫 벡터 텐서 $\rightarrow [N, S, d]$ 크기의 임베딩 텐서
- 변환된 임베딩 텐서는 트랜스포머의 모든 계층에서 공유되는 입력 텐서가 됨

트랜스포머 인코더

입력 시퀀스 $\rightarrow$ 인코더 계층 (=멀티 헤드 어텐션+순방향 신경망) - 입력 데이터 정보 추출
 $\rightarrow$ 다음 인코더 계층 입력 $\rightarrow \dots \rightarrow$ 인코더 출력 $\Rightarrow$ 디코더 계층으로 전달

인코더 계층 연산

- 목적: 입력 벡터 + 위치 임베딩 벡터 합

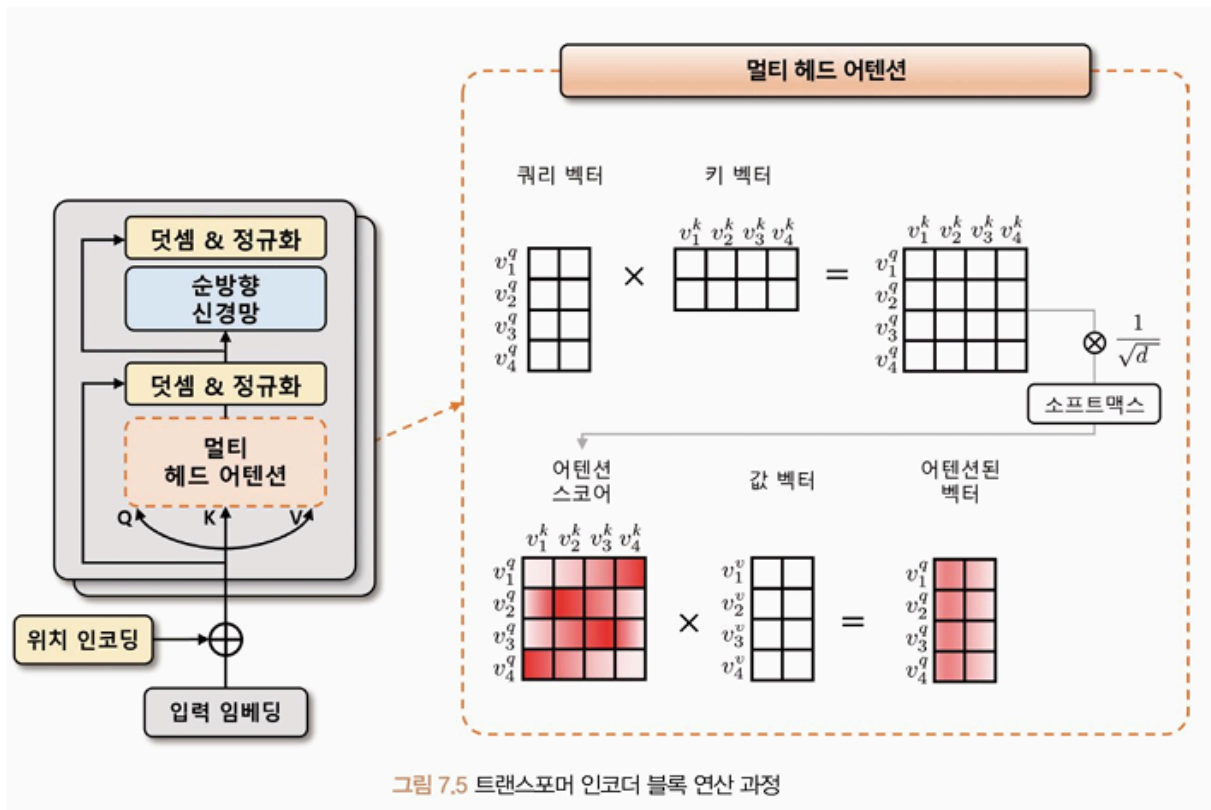


그림 7.5 트랜스포머 인코더 블록 연산 과정

1. **입력**: 소스 데이터 입력 임베딩 [N, S, d]
2. **멀티 헤드 어텐션 단계**: 선형 변환 → 3개 임베딩 벡터: 쿼리(Q), 키(K), 값(V) 벡터 생성
 - 쿼리 벡터(v^q): 내가 지금 뭘 알고 싶은지
 - 현재 시점에서 참조하고자 하는 다른 시점의 정보의 위치를 나타내는 벡터. 인코더 각 시점마다 생성됨.
 - 키 벡터(v^k): 각 단어가 어떤 정보를 가지고 있는지
 - 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터. 인코더 각 시점마다 생성됨.
 - 값 벡터(v^v): 그 단어의 실제 정보
 - 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

→ 쿼리, 키, 값 벡터는 초기에 비슷한 값을 가지다가 모델 학습을 통해 의도한 의미의 값을 갖게 됨

수식 7.2 어텐션 스코어 계산식

$$\text{score}(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- **셀프 어텐션 과정**

1. 쿼리, 키, 값 내적 = 어텐션 스코어
2. 벡터 차원의 제곱근만큼 나눠 보정 (벡터 차원이 커질 때 스코어값이 같이 커지는 문제 완화)
3. 소프트맥스 함수 → 확률적 재표현
4. 값 벡터와 내적 = 셀프 어텐션된 벡터

1~4 반복 → 셀프 어텐션된 스코어 맵 생성

- **멀티 헤드(multi-head) 어텐션**: 셀프 어텐션 여러 번 수행 → 여러 개의 헤드 생성. 각 헤드는 독립적으로 어텐션을 수행하고 그 결과를 합침.

- 즉 $[N, S, d]$ 입력 텐서에 k 개의 셀프 어텐션 벡터 생성할 때 헤드에 대한 차원 축을 생성해 $[N, k, S, d/k]$ 텐서 형태를 구성함.

= k 개의 셀프 어텐션된 $[N, S, d/k]$ 텐서

- → 임베딩 차원 축으로 **병합**(concatenation) → $[N, S, d]$ 형태로 출력됨

3. **덧셈 & 정규화**

- 덧셈: 멀티 헤드 어텐션 입력값 $[N, S, d]$ 텐서 + 출력값 $[N, S, d]$ 텐서 ⇒ 학습 시 발생하는 기울기 소실 완화

- 정규화: 임베딩 차원 축으로 계층 정규화

4. **순방향 신경망**: 선형 임베딩 + ReLU 또는 Conv1d

5. **덧셈 & 정규화**

- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성됨. 벡터가 여러 블록을 거치면서 점차 입력 시퀀스의 정보가 추상화됨.
- 마지막 인코더 블록의 출력 벡터 → 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터

트랜스포머 디코더

타깃 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD], [PAD], [PAD], [PAD]

디코더 계층 연산

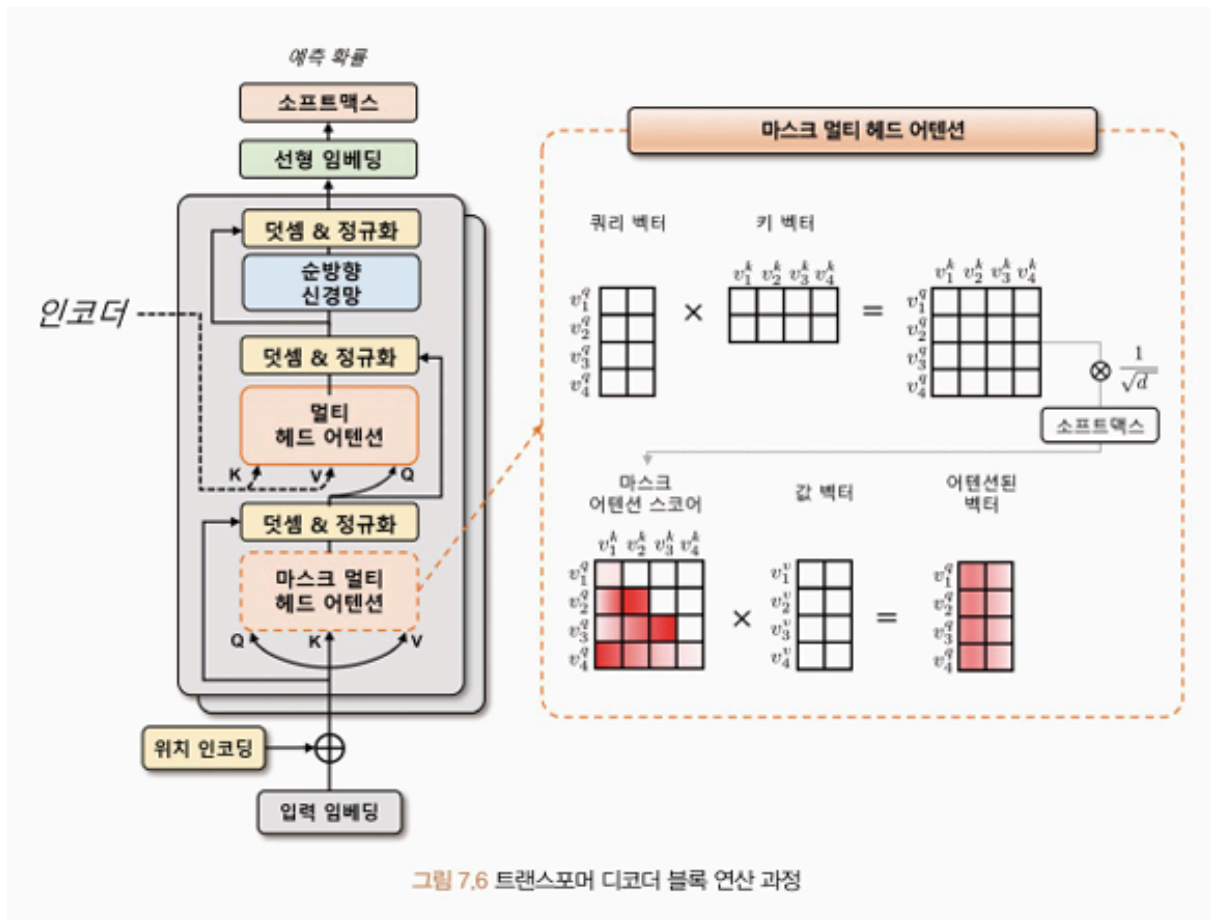


그림 7.6 트랜스포머 디코더 블록 연산 과정

1. 입력: 타깃 데이터 입력 임베딩

2. 마스크 멀티 헤드 어텐션

- 인과성(Casuality)이 반영된 멀티 헤드 어텐션 모듈
 - 어텐션 스코어 맵을 계산할 때 $v^q_1 \rightarrow v^k_1, v^q_2 \rightarrow v^k_1, v^k_2$ 를 바라보도록 마스크를 씌움
 - 마스크 적용 → 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되어 인과성이 보장됨
- **마스크 영역**에 $-\text{inf}$ 마스크 더함 → 해당 영역 어텐션 스코어 값이 0에 가까워짐
- 소프트맥스 계산 → 해당 영역 어텐션 가중치가 0이 됨 → 마스크 영역 이전에 있는 입력 토큰들에 대한 정보 참조 X

3. 멀티 헤드 어텐션

- 타깃 데이터 입력 임베딩 + 위치 인코딩 합한 벡터 → 쿼리 벡터
인코더의 소스 데이터 → 키, 값 벡터
- 어텐션 스코어 맵 계산

- 소프트맥스 함수 → 어텐션 가중치 계산
- 어텐션 가중치와 값 벡터 가중합 ⇒ 멀티 헤드 어텐션 출력 벡터

+) 나머지...

- 이전 트랜스포머 디코더 블록의 산출물 → 다음 블록 입력
- 이전 시점 정보를 현재 시점에서 활용하는 것
- 마지막 디코더 블록 출력 텐서 $[N, S, d]$ → 선형 변환, 소프트맥스 ⇒ 각 타깃 시퀀스 위 차별 예측 확률

실습

Multi30k 데이터셋, 파이토치 트랜스포머 모델로 영어-독일어 번역 모델 구성

- Multi30k: 영어-독일어 병렬 말뭉치(Parallel corpus), 30,000개 약 데이터 제공

387p

GPT (Generative Pre-trained Transformer)

- 2018 OpenAI에서 발표한 트랜스포머 기반 언어 모델
- ELMo(Embeddings from Language Model) 등 대규모 말뭉치로 사전 학습된 모델
- 다양한 다운스트림 작업에서 우수한 성능을 보임
- 트랜스포머의 디코더를 여러 층 쌓아 만든 모델
 - 인코더: 입력 문장 특징 추출에 초점
 - 디코더: 입력 문장, 출력 문장 간 관계 이해 및 출력 문장 생성에 초점
- GPT 시리즈

	매개변수 (모델 크기)	특징
GPT-1	약 1억 2천만 개	
GPT-2	약 15억 개	
GPT-3	약 1790억 개	

GPT-3.5 (InstructGPT)	약 60억 개	RLHF(Reinforcement Learning from Human Feedback) 도입 : 인간의 피드백을 이용하는 강화 학습 방법 GPT-3.5 기반 ChatGPT 서비스 공개
GPT-4		이미지 데이터 처리 가능

+) GPT-Neo, GPT-J 등

GPT-1

트랜스포머 구조를 바탕으로 한 단방향(undirectional) 언어 모델

→ 이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작

→ 모델 계산량 감소 ⇒ 학습 속도 증가, 모델 크기 감소

구조

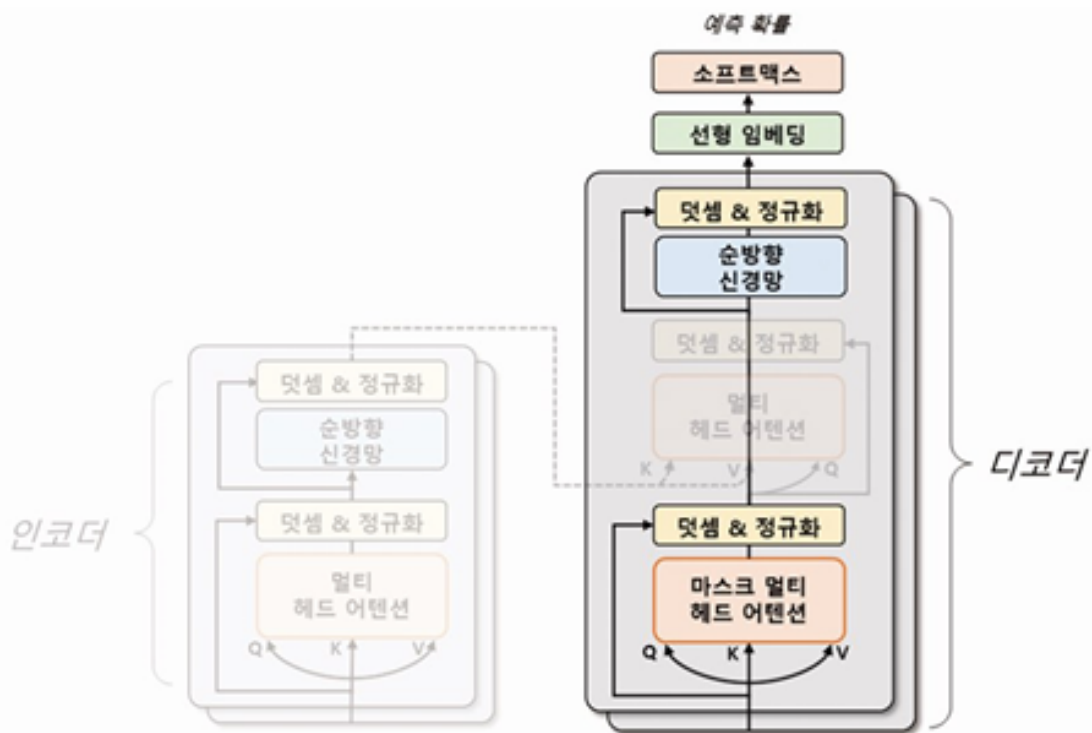
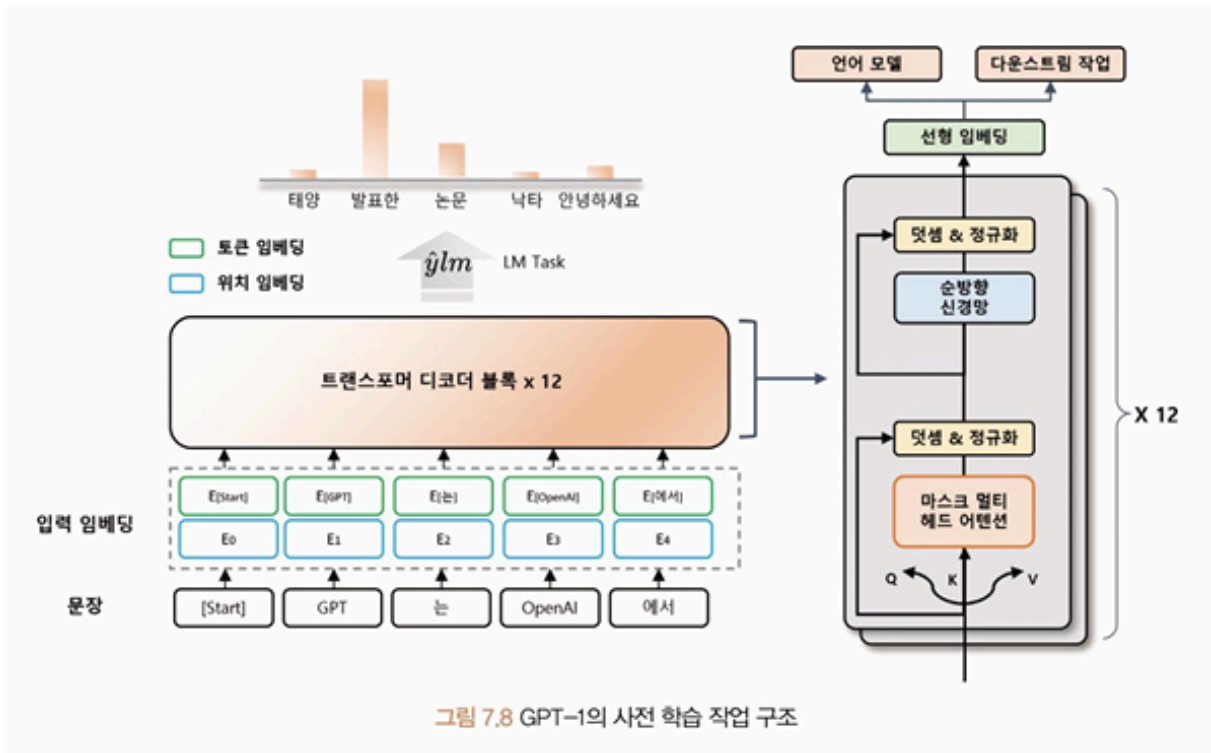


그림 7.7 트랜스포머와 GPT-1 모델 비교

- 트랜스포머 구조 중 디코더 부분만 사용
- 디코더의 두 번째 멀티 헤드 어텐션 사용 X
- 12개의 헤드를 가진 트랜스포머 디코더 12층 → 약 1억 2천만 개 매개변수로 학습

- 약 4.5GB의 BookCorpus 데이터셋을 사용해 언어 모델 사전 학습
 - 입력 문장을 임의의 위치까지만 보여줌 → 모델이 뒤에 이어지는 문장 예측
 - 비지도 학습 → 컴퓨팅 자원만 충분하면 제약 없이 학습 가능

사전 학습



입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델로 사전 학습

- 문장을 토큰으로 나누어 처리
 - 입력 문장 → 일정한 길이의 블록으로 나누어 처리
 - 각 블록 → 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰 예측
 → 장기적 문맥 이해 가능
- 입력 임베딩 계산: 토큰 임베딩 + 위치 임베딩
 - 토큰 임베딩: 토큰 → 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터 매핑
- 다운스트림 작업 구조: 미세 조정을 위한 다운스트림 작업에서, 각 작업에 따라 입출력 구조를 바꾸지 않고 언어 모델을 **보조 학습(Auxiliary Learning)**해 사용 가능

- 보조 학습: 모델이 주어진 주요 학습 작업 외에도 부가적 학습 작업을 동시에 수행하는 것

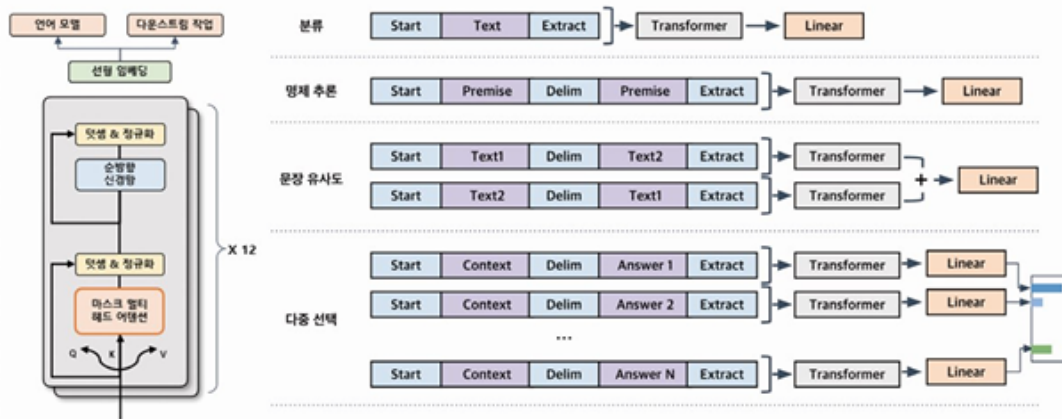
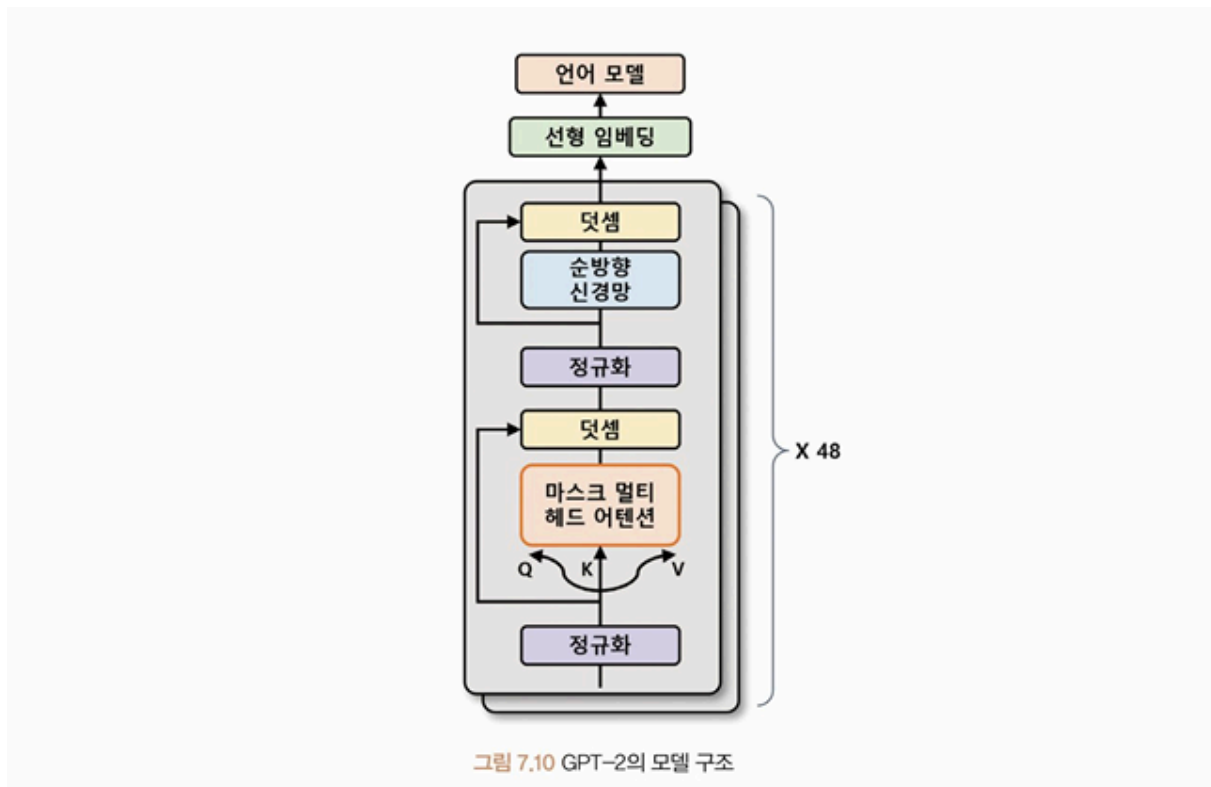


그림 7.9 GPT-1의 다운스트림 작업 구조

- 2개의 손실 함수로 최적화: 일반 언어 모델 학습용 손실 함수 + **다운스트림 작업용 추가 손실 함수** 필요
(원하는 목표에 맞게 모델을 세밀하게 조정해야 하기 때문)
- 특수 토큰 사용: <start>, <delim>, <extract>

GPT-2

	GPT-1	GPT-2
입력 문장 최대 길이	512	1024
디코더 층	12	48 → 더 복잡한 패턴 학습 가능
매개변수	1200만 개	15억 개
학습 데이터셋 크기	4.5GB BookCorpus	40GB 웹사이트 스크래핑 약 8백만개 문서
		제로-샷 학습 가능 : 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있도록 사전 학습 과정에서 특정 형식 데이터 입력 ⇒ 배우지 않은 작업에 사용 가능

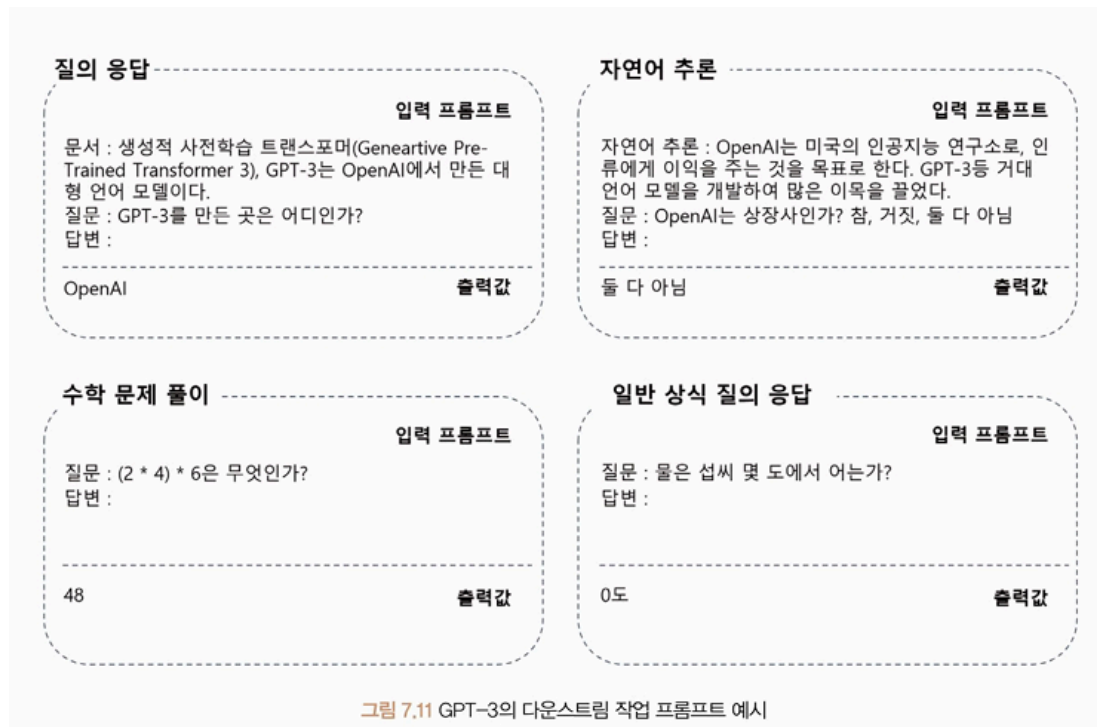


GPT-3

- GPT-2보다 약 116배 많은 매개변수
- 멀티 헤드 어텐션 헤드 96개
→ 연산량 줄이기 위해 희소 어텐션(Sparse Attention), 일반 어텐션 섞어 사용
- 트랜스포머 디코더 96층
- 토큰 임베딩 크기 1,600 → 12,888
 - 토큰 입력 최대 크기 - GPT-2: 1,024개 / GPT-3: 2,048개
- 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB 데이터셋 이용해 모델 학습

장점

- 다운스트림 작업에서 높은 성능을 보임
 - 프롬프트(prompt) 형식으로 작업 수행

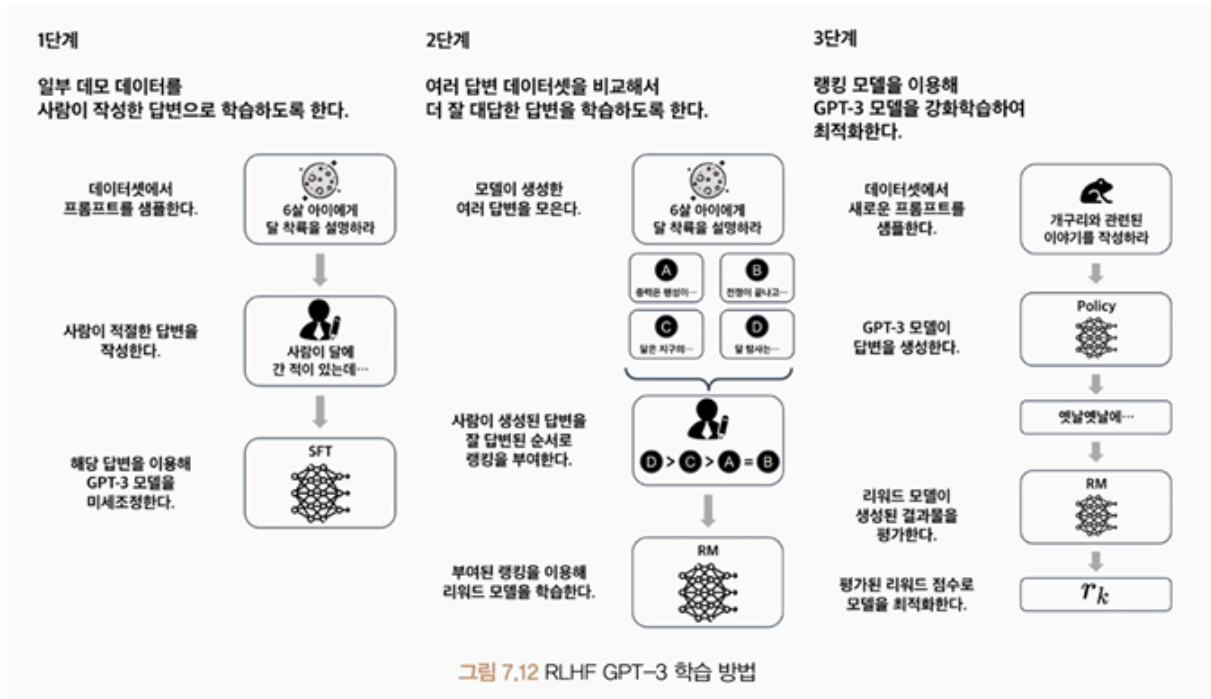


- 새로운 도메인 작업에서 높은 일반화 능력, 기존 자연어 처리 분야에서 기술적 한계 극복 단점
- 느리고 비용이 많이 듦 → 일반 사용자, 소규모 기업에서 활용하기 어려움
- 사전 학습된 데이터에 기반 → 데이터가 편향되어 있거나 정확하지 않으면 오류 발생
- 사람에 비해 인간적인 이해력, 상식적 추론 능력 부족
- 신뢰성 부족, 편견 및 혐오 표현 포함 가능성 O

GPT-3.5 (InstructGPT)

- GPT-3 모델 구조 + RLHF 도입 → 매개변수 수 감소, 모델 자연스러움 증가

RLHF GPT-3 학습 방법



1. 일부 데모 데이터(프롬프트(State))에 대해 사람이 작성한 답변으로 학습
2. 답변 데이터셋에 랭킹 부여, 리워드 모델 학습
3. **지도 미세 조정(Supervised Fine-Tuning, SFT):** 리워드 모델을 이용해 GPT 모델 (Policy)이 생성(Action)한 답변 평가 → (Reward) 모델이 생성한 문장이 자연스럽다면 보상받고, 그렇지 않으면 보상받지 못함 = 강화학습
 - GPT 모델은 같은 프롬프트에 대해서도 시드(seed)에 따라 다른 답변 생성하기 때문에 가능
 - 지도 미세 조정 과정에서 생성된 답변과 랭킹 정보를 활용해 리워드 모델 학습 → 더 자연스러운 문장 생성하는 GPT-3.5 학습
 - 이전 모델에 비해 성능 향상
 - 환각(Hallucination) 현상 여전히 존재: 언어 모델이 일부 입력에 대해 현실적이지 않은 결과 생성. 인간같은 추론, 판단 능력을 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상

GPT-4

- 이미지 데이터도 인식 가능한 멀티모달(Multimodal) 모델

	GPT-3.5	GPT-4
처리 토큰 개수	4096	32768

처리 단어 개수	3000	25000
대규모 다중 작업 언어 이해 (Massive Multitask Language Understanding, MMLU) 벤치마크 테스트 결과	70%	85% (한국어: 77%)
미국 변호사 시험	213점	298점
SAT 읽기, 쓰기 시험	670점	710점

- 모델 매개변수 수, 학습 데이터 및 방법 공개 X
- ChatGPT Plus, 마이크로소프트 Bing으로 모델 사용 가능

모델 실습

허깅 페이스 트랜스포머 라이브러리 GPT-2 모델로 문장 생성

```
# GPT-2를 이용한 문장 생성
from transformers import pipeline

generator = pipeline(task="text-generation", model='gpt2')
## 입력된 작업에 모델로 적합한 파이프라인 구축
## pipeline: 다양한 작업에 대한 전처리, 모델 아키텍처, 후처리 각각 처리하는 함수
outputs=generator(
    text_inputs="Machine learning is",
    max_length=20,
    num_return_sequences=3,
    pad_token_id=generator.tokenizer.eos_token_id # end of sentence
)

print(outputs)
```

- `pipeline`: task-입력 작업, model-모델 로 적합한 파이프라인 구축
 - task: 수행하려는 작업 ex) `text-generation`
 - model: 사용 모델 ex) `'gpt2'`
 - 설정한 작업 수행하는 파이프라인 클래스 인스턴스 반환
 - ex) `TextGenerationPipeline` 클래스의 인스턴스 생성
- `generator` 함수

- `text_inputs` : 생성하려는 문장 입력 문맥 전달
- `max_length` : 생성될 문장의 최대 토큰 수
- `num_return_sequence` : 반환 시퀀스 개수
- `pad_token_id` : 패딩 토큰 ID. 모델의 자유 생성(Open-end generation) 여부 설정.

CoLA(The Corpus of Linguistic Acceptability) 데이터셋 로드

```
import torch
```

```
from torchtext.datasets import CoLA
```

```
from transformers import AutoTokenizer
```

```
from torch.utils.data import DataLoader
```

collator: 여러 설정 작업 수행

```
def collator(batch, tokenizer, device):
```

```
    source, labels, texts = zip(*batch)
```

```
    tokenized = tokenizer(
```

```
        texts,
```

```
        padding="longest", # 패딩 - 가장 긴 시퀀스에 패딩 적용
```

```
        truncation=True, # 절사 - 최대 길이 초과 시 시퀀스 자르기
```

```
        return_tensors='pt', # 반환 형식 설정 - Tensor
```

```
    )
```

```
    input_ids = tokenized["input_ids"].to(device)
```

```
    attention_mask = tokenized["attention_mask"].to(device)
```

```
    labels = torch.tensor(labels, dtype=torch.long).to(device)
```

토큰 ID, 어텐션 마스크 반환

```
    return input_ids, attention_mask, labels
```

토큰 id: 각 토큰에 대해 부여한 숫자 ID

어텐션 마스크: 입력 문장의 단어 - 1 / 패딩 - 0 으로 채움

CoLA → train, dev, test 데이터셋으로 구성됨

```
train_data = list(CoLA(split="train")) # 학습
```

```
valid_data = list(CoLA(split="dev")) # 검증
```

```
test_data = list(CoLA(split="test")) # 평가
```

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

```
tokenizer.pad_token = tokenizer.eos_token # gpt-2는 패딩 토큰이 따로 없음
```

```

epochs=3
batch_size=16
device="cuda" if torch.cuda.is_available() else "cpu"

# 데이터로더 설정
train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
    shuffle=True,
)
valid_dataloader = DataLoader(
    valid_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokeni
zer, device)
)
test_dataloader = DataLoader(
    test_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokeniz
er, device)
)

print("Train Dataset Length: ", len(train_data))
print("Valid Dataset Length: ", len(valid_data))
print("Test Dataset Length: ", len(test_data))

```

```

# GPT-2 모델 설정
from torch import optim
from transformers import GPT2ForSequenceClassification # 문장 분류 모델
클래스

model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2 # CoLA 데이터셋은 올바른 문장과 올바르지 않은 문장이 태깅되어
있음
).to(device)
model.config.pad_token_id = model.config.eos_token_id # 패딩 토큰 설정
optimizer = optim.Adam(model.parameters(), lr=5e-5)

```

- `GPT2ForSequenceClassification` : GPT-2 모델을 기반으로 하는 시퀀스 분류 모델. 기본 GPT-2 모델과 유사하게 작동하는데, 최종 출력 계층이 분류를 위해 미세 조정됨
- 문장 분류 모델을 학습하기 위해서 모델이 고정된 길이의 입력을 필요로 함 → 패딩 토큰 설정 필요

GPT-2 모델 학습 및 검증

```
import numpy as np
from torch import nn
```

```
def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat) # 정확도 반환
```

```
def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0
```

```
    for input_ids, attention_mask, labels in dataloader:
        outputs = model( # 데이터 -> 모델
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
```

```
        loss = outputs.loss # 학습 시 내부적으로 손실 계산 및 반환
        train_loss += loss.item()
```

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
    train_loss = train_loss / len(dataloader)
    return train_loss
```

```
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
```

```

criterion = nn.CrossEntropyLoss()
val_loss, val_accuracy = 0.0, 0.0

for input_ids, attention_mask, labels in dataloader:
    outputs = model( # 평가용 데이터 -> 모델
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )
    logits=outputs.logits # 확률

    loss = criterion(logits, labels)
    logits=logits.detach().cpu().numpy()
    label_ids=labels.to("cpu").numpy()
    accuracy = calc_accuracy(logits, label_ids) # 확률, 레이블로 손실 계산

    val_loss+=loss
    val_accuracy+=accuracy

val_loss = val_loss / len(dataloader)
val_accuracy = val_accuracy / len(dataloader)
return val_loss, val_accuracy

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "/content/drive/MyDrive/euron/datasets/Week12/GPT2ForSequenceClassification.pt")
        print("Saved the model weights")

```

- 결과


```
Epoch 1: Train Loss: 0.6704 Val Loss: 0.6041 Val Accuracy 0.6910
Saved the model weights
Epoch 2: Train Loss: 0.5870 Val Loss: 0.5710 Val Accuracy 0.7061
Saved the model weights
Epoch 3: Train Loss: 0.5069 Val Loss: 0.5264 Val Accuracy 0.7383
Saved the model weights
```

- epochs=3 으로 학습 및 평가
- 검증 손실값이 가장 낮은 값을 가지는 체크포인트 저장
- 학습 과정에서 최적의 모델이 저장됨

모델 평가

```
model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2
).to(device)
model.config.pad_token_id=model.config.eos_token_id
model.load_state_dict(torch.load("/content/drive/MyDrive/euron/datasets/
Week12/GPT2ForSequenceClassification.pt"))

test_loss, test_accuracy = evaluation(model, test_dataloader)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy:.4f}")
```

- 결과
 - 허깅페이스에 있는 데이터셋으로 했는데 test_data의 labels=-1 인 문제로 오류가 나서 결과가 나오지 않음. CoLA 데이터셋을 사용해보려다가 실패!
 - 학습 데이터셋이 8500개로 모델 규모에 비해 매우 작은 데이터셋을 사용했다는 걸 감안하면 모델의 성능이 매우 높다고 볼 수 있음